

Week 3 教材ノート

zkSNARK を理解するための前提知識

教材ドラフト

2026 年 7 月 1 日

目次

1	このノートの使い方	4
1.1	Week 3 の到達目標	4
1.2	全体の見取り図	4
1.3	このノートで使う記法	5
2	zkSNARK の数式を使わない導入	6
2.1	証明者と検証者	6
2.2	ウォーリーを探せの直感	6
2.3	SNARK の主な性質	7
2.4	zkSNARK の部品一覧	9
2.5	statement を数式で書く	9
2.6	正しさ、知識、秘匿性を分ける	10
2.7	離散対数の知識証明	10
3	有限体、多項式、FFT	12
3.1	有限体の直感	12
3.2	体とは何か	13
3.3	逆元と割り算	13
3.4	拡張 Euclid 互除法で逆元を求める	13
3.5	なぜ有限体を使うのか	14
3.6	有限体上で制約を書く	14
3.7	巡回群と位数	15
3.8	DL、CDH、DDH	16
3.9	多項式は制約の共通言語	16
3.10	補間を式で見る	16
3.11	多項式が等しいことをどう確認するか	17
3.12	vanishing polynomial	17

3.13	FFT は何を高速化するのか	18
4	楕円曲線	19
4.1	点を足せる世界	19
4.2	楕円曲線を式で見る	19
4.3	点の足し算の式	20
4.4	ねじれ点 $E[m]$	21
4.5	スカラー倍はなぜ速いか	21
4.6	離散対数問題	21
4.7	ペアリング	22
5	Commitment Scheme	23
5.1	commitment の基本	23
5.2	Merkle tree	25
5.3	Merkle proof を式で追う	26
5.4	Pedersen commitment	27
5.5	Pedersen の hiding と binding を式で見る	27
5.6	KZG commitment	28
5.7	KZG opening を式で見る	28
5.8	比較表	30
5.9	zkSNARK での役割	30
6	Arithmetization	30
6.1	arithmetization の目的	31
6.2	プログラムを制約にするとどういうことか	31
6.3	R1CS	31
6.4	AIR	33
6.5	AIR を多項式制約にする	33
6.6	Plonkish arithmetization	34
6.7	Plonkish の gate を式で見る	35
6.8	copy constraint と permutation argument	36
6.9	lookup を式の気持ちで理解する	37
6.10	CCS	38
6.11	CCS の定義	38
6.12	R1CS は CCS の特別な場合	39
6.13	Plonkish 風の gate も CCS で見る	39
6.14	CCS が folding で役立つ理由	40
6.15	比較表	41
6.16	同じ計算を複数の見方で見ると	41
7	講義内チェックリスト	42

8	ホワイトボードセッション	42
8.1	目的	42
8.2	推奨進行	42
8.3	グループ内の役割	42
8.4	1 枚 HTML レポートの構成	43
8.5	ピアレビュー観点	43
9	実装課題の選び方	43
9.1	各 README に入れる項目	44
9.2	課題 A: 不十分な KZG protocol に対する攻撃	44
9.3	課題 B: Ethereum blob で使われる commitment scheme を調べる	44
9.4	課題 C: 3 次方程式の解を知っていることを証明する回路を書く	45
9.5	課題 D: 簡易ステートマシンを AIR で実装する	45
9.6	課題 E: Arithmetization API のプロトタイプ	45
10	用語集	45
11	Further Reading	46
11.1	入門	46
11.2	有限体、多項式、FFT	46
11.3	Commitment Scheme	46
11.4	Arithmetization	46
12	確認問題の解答例	47
12.1	zkSNARK 導入	47
12.2	有限体、多項式、FFT	47
12.3	楕円曲線	47
12.4	Commitment Scheme	47
12.5	Arithmetization	47

1 このノートの使い方

このノートは、zkSNARK を初めて体系的に学ぶ参加者が、講義後にも何度も見返せることを目的にした教材である。有限体、楕円曲線、commitment scheme、arithmetization を個別の用語として暗記するよりも、「計算をどう証明可能な形に変換し、大きな対象をどう短い証明に圧縮するのか」という一つの流れとして読む。

1.1 Week 3 の到達目標

- zkSNARK の全体像を、数式に入る前の直感として説明できる。
- 有限体、多項式、FFT が、なぜ zk の基礎言語になるのかを説明できる。
- Merkle tree、Pedersen commitment、KZG commitment の性質、用途、トレードオフを比較できる。
- R1CS、AIR、Plonkish arithmetization、CCS を、計算や制約を表現する方法として比較できる。

1.2 全体の見取り図

現代的な多くの zkSNARK や多項式 commitment ベースの証明システムは、大づかみに言えば次のパイプラインで理解できる。

計算 $\rightarrow$ 制約 $\rightarrow$ 多項式 $\rightarrow$ commitment $\rightarrow$ proof

このパイプラインの各段階で、異なる数学や暗号の道具が登場する。有限体は計算の舞台を作る。多項式は制約や実行履歴をまとめて扱う言語になる。commitment scheme は、大きなデータや多項式を短い値で固定する。arithmetization は、プログラムや計算を有限体上の制約へ翻訳する。

もう少し形式的に書くと、証明したい対象は多くの場合、ある関係 R として表される。公開入力を x 、witness を w とすると、

$$R(x, w) = 1$$

であることを示したい。ここで R は「検査プログラム」と読める。例えば、

$$R(y, x) = 1 \iff x^3 + x + 5 = y$$

なら、公開入力は y 、witness は x である。zkSNARK が目指すのは、検証者に x を渡さずに、短い proof π だけで

$$\text{Verify}(y, \pi) = 1$$

と確認できるようにすることである。つまり、大きな計算 R の再実行を、短い検証手続きへ置き換える。

■まず覚えること

- まず「何を証明したいのか」を公開された主張として切り出す。
- 次に「なぜそれが正しいのか」を秘密情報や計算履歴として持つ。
- 最後に、その秘密を保ったまま、短く検証できる証明へ圧縮する。

1.3 このノートで使う記法

このノートでは、暗号の基礎部分でよく使う記法をそろえておく。特に、有限体、群、楕円曲線、ペアリングでは、同じ概念が乗法記法と加法記法で表れるため、最初に対応を固定しておく。

記号	意味
$\mathbb{Z}/m\mathbb{Z}$	整数を m で割った余りの集合。一般には剰余環。
$\mathbb{F}_p$	p 個の元 $\{0, 1, \dots, p-1\}$ からなる有限体。ここでは p は素数。
$\mathbb{F}_p^*$	$\mathbb{F}_p$ から 0 を除いた集合。掛け算について群になる。
$\langle g \rangle$	元 g が生成する巡回群。乗法記法では $\{g^0, g^1, g^2, \dots\}$ 。
E/k	体 k 上の楕円曲線。典型的には $y^2 = x^3 + ax + b$ と無限遠点 O からなる。
$E[m]$	楕円曲線 E の m 等分点、またはねじれ点の集合。 $E[m] = \{P \in E \mid mP = O\}$ 。
e	ペアリング。典型的には $e: G_1 \times G_2 \rightarrow G_T$ の双線形写像。

暗号では、同じ数学的構造を二通りの記法で書くことが多い。有限体の非ゼロ要素や一般の巡回群では、乗法記法を使って

$$g^a, \quad g^{ab}$$

のように書く。一方、楕円曲線上の点の群では、加法記法を使って

$$aP, \quad abP$$

のように書く。対応関係は次の通りである。

概念	乗法群の記法	楕円曲線など加法群の記法
生成元	g	P
秘密スカラーを掛けた公開値	g^a	aP
共有値	g^{ab}	abP
離散対数問題	g, g^a から a を求める	P, aP から a を求める

この対応を意識すると、有限体上の Diffie–Hellman、楕円曲線 Diffie–Hellman、Pedersen commitment、KZG commitment が同じ「スカラーは隠し、群要素だけを公開する」構造を共有していることが見えやすくなる。

2 zkSNARK の数式を使わない導入

■この章で答える問い

- 「証明する」とは何をすることか。
- 「秘密を明かさずに証明する」とはどういうことか。
- zkSNARK の学習で、なぜ数学、暗号、コンパイラ的な話が同時に出てくるのか。

2.1 証明者と検証者

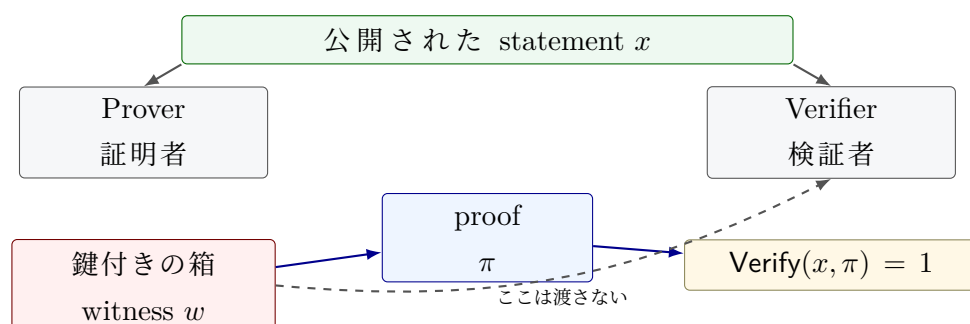
zkSNARK では、主に二つの役割を考える。

証明者 ある主張が正しいことを示したい人。witness や計算過程を知っている。

検証者 その主張が正しいかを確認したい人。秘密そのものを受け取らずに確認したい。

証明者が持っている補助入力を **witness** と呼び、検証者にも公開される主張を **statement** と呼ぶ。zk の文脈では witness を秘密にしたいことが多いが、用語としての witness は「主張を成り立たせるための補助情報」を指す。例えば「公開値 y に対して、 $x^3 + x + 5 = y$ を満たす秘密の x を知っている」という主張では、 y が statement、 x が witness である。

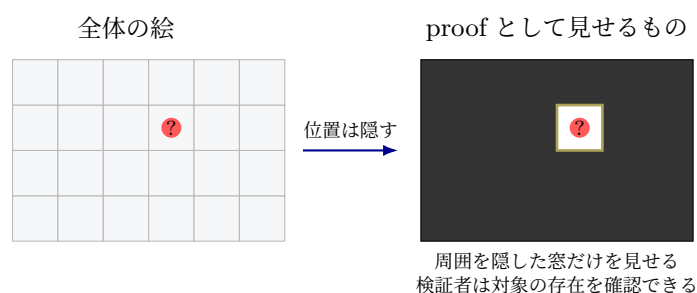
Prover と Verifier の見える関係



2.2 ウォーリーを探せの直感

ウォーリーの場所を知っていることを示すには、場所そのものを教えればよい。しかし、それでは秘密が漏れてしまう。zero-knowledge の直感は、「場所を直接教えずに、たしかに知っている」と納得してもらおう」ことにある。

場所を見せずに「知っている」と納得させる



実際の zkSNARK では、数学的な制約と暗号的な commitment を使う。ただし、直感としては次の対応を持っておくとよい。

用語	直感
statement	公開された問題。「この絵の中にウォーリーがいる」
witness	主張を成り立たせる補助情報。「ウォーリーの場所」
proof	答えを直接見せずに、知っているのと納得させる証拠
verifier	証拠を短時間で確認する人またはプログラム

2.3 SNARK の主な性質

SNARK は、**Succinct Non-interactive Argument of Knowledge** の略である。ここで argument とは、計算量的に制限された不正な証明者に対して安全性を考える証明体系を指す。また、of knowledge は「有効な proof を作れるなら、対応する witness を知っているはずだ」という知識健全性の要件を表す。

Completeness 正しい witness を持つ正直な証明者の proof が受理されること。

Knowledge soundness 有効な proof を作れるなら、対応する witness を取り出せること。

Zero-knowledge witness について、statement の正しさ以上の情報を漏らさないこと。

Succinctness proof が短く、検証が速いこと。

Non-interactive 証明者と検証者が何度も対話しなくてよいこと。

■注意

zero-knowledge で検証者が学ぶ内容は、statement の正しさに限られる。witness や、そこから推測できる余計な情報は秘匿する。

■発展内容

数学的には、まず関係

$$R \subseteq \{0,1\}^* \times \{0,1\}^*$$

を固定する。公開入力 x と witness w が $(x, w) \in R$ を満たすとき、 w は x の正しい witness である。この関係から定まる言語を

$$L_R = \{x \mid \exists w \text{ such that } (x, w) \in R\}$$

と書く。ZKP は、この $x \in L_R$ という主張を、 w を明かさずに納得させるための証明プロトコルである。

セキュリティパラメータを λ とし、無視できる関数を $\text{negl}(\lambda)$ と書く。これは任意の正の多項式 p に対して、十分大きな λ で

$$\text{negl}(\lambda) < \frac{1}{p(\lambda)}$$

となる関数を意味する。このノートでは SNARK を見据え、健全性は最初から knowledge soundness として述べる。対話型プロトコル $\Pi = (P, V)$ が関係 R に対する zero-knowledge argument of knowledge であるとは、典型的には次を満たすことをいう。

Completeness 任意の $(x, w) \in R$ について、正直な証明者 P と検証者 V が実行すると、

$$\Pr[\langle P(w), V \rangle(x) = 1] \geq 1 - \text{negl}(\lambda)$$

となる。

Knowledge soundness 任意の効率的な不正証明者 P^* に対して、効率的な抽出器 Ext^{P^*} が存在する。証明者が x について検証者を受理させる確率を

$$\alpha_{P^*}(x) := \Pr[\langle P^*, V \rangle(x) = 1]$$

とおく。このとき、抽出器は同じ証明者の振る舞いから witness を取り出し、

$$\Pr[w \leftarrow \text{Ext}^{P^*}(x) : (x, w) \in R] \geq \alpha_{P^*}(x) - \text{negl}(\lambda)$$

を満たす。つまり、受理される proof を高い確率で作れる証明者からは、対応する witness もほぼ同じ確率で取り出せる。実際の定義では、抽出器が証明者を巻き戻すか、共通参照文字列を使うかなどに応じて実験の細部を定める。

Zero-knowledge 任意の効率的な不正検証者 V^* に対して、効率的なシミュレータ S が存在し、

$$\text{Real}_{P, V^*}(x, w) \stackrel{c}{\approx} \text{Sim}_{S, V^*}(x)$$

が成り立つ。左辺は本物の実行で検証者が得る記録、右辺は witness を使わずに生成した擬似的な記録である。この記録には、公開入力、検証者の乱数、送受信したメッセージ、最終的な受理または拒否の結果が含まれる。文献ではこの記録を view と呼ぶことが多い。二つの記録が計算量的に区別できないなら、検証者は witness 由来の追加情報を得ていないと考える。

SNARK のような非対話型 ZKP では、会話は一つの proof π に圧縮される。この場合の knowledge soundness は、受理される π を作る証明者から、抽出器が $R(x, w) = 1$ を満たす w を取り出せる、という形で読む。少し形式的には、任意の効率的な証明生成アルゴリズム A に対して抽出器 Ext^A が存在し、

$$\Pr[\text{Verify}(\text{vk}, x, \pi) = 1 \text{ and } (x, w) \notin R : (x, \pi, w) \leftarrow \text{KExp}_{A, \text{Ext}^A}(\text{pp})] \leq \text{negl}(\lambda)$$

と書ける。ここで KExp は、同じ公開パラメータ pp のもとで A から statement と proof を得て、抽出器から witness を得る実験を表す。その場合の zero-knowledge 性は、実 proof の分布

$$\{\pi \leftarrow P(x, w)\}$$

と、シミュレータが witness なしで作る分布

$$\{\pi \leftarrow S(x)\}$$

が計算量的に区別できない、という形で読むとよい。ここでいう「同じ」は、proof の文字列が毎回一致するという意味ではない。証明生成には乱数が入ることがあり、同じ statement と

witness から複数の異なる proof が出てもよい。重要なのは、実 proof と simulated proof を見分ける効率的な検証者が存在しないことである。perfect zero-knowledge では分布が完全に一致し、statistical zero-knowledge では統計的に近く、computational zero-knowledge では計算量的に区別できない。

2.4 zkSNARK の部品一覧

zkSNARK の内部では、次のような部品がつながっている。

- 計算を制約に変換する: arithmetization
- 制約を多項式の主張に変換する: polynomial view
- 多項式や大量データを短い値に固定する: commitment scheme
- 短い proof を生成し、高速に検証する: proof system

■小例: 3 次方程式の秘密

公開値 $y = 15$ がある。証明者は、 $x^3 + x + 5 = 15$ を満たす $x = 2$ を知っている。検証者に x を教えず、「そのような x を知っている」ことだけを示したい。

このとき、statement は「 $y = 15$ に対して条件を満たす x が存在する」、witness は $x = 2$ である。

2.5 statement を数式で書く

暗号プロトコルの説明では、「ある条件を満たす秘密を知っている」という言い方がよく出てくる。これは次のように書ける。

$$\text{statement: } \exists w \text{ such that } R(x, w) = 1$$

ここで x は公開入力、 w は witness、 R は検査したい関係である。 $\exists$ は「存在する」という意味である。検証者が確認したい内容は、「条件を満たす w が少なくとも一つ存在するか」である。

■小例: 数式を statement に分解する

「 $y = 15$ に対して $x^3 + x + 5 = y$ を満たす x を知っている」は、

$$\exists x \in \mathbb{F} \text{ such that } x^3 + x + 5 = 15$$

と書ける。証明者は $x = 2$ を知っている。検証者は x を受け取らずに、この存在主張が正しいことを確認したい。

ここで重要なのは、検証者が最終的に確認する対象は

「方程式を満たす解がある」

である。

「解は 2 である」

という情報までは渡さない。

2.6 正しさ、知識、秘匿性を分ける

zkSNARK では、似た言葉が同時に出るため、次の三つを分けて考えるとよい。

正しさ statement が本当に成り立つこと。

知識 証明者が、statement を成り立たせる witness を知っていること。

秘匿性 検証者が witness そのものを学ばないこと。

先ほどの例では、正しさは「方程式に解がある」こと、知識は「証明者がその解を知っている」こと、秘匿性は「検証者が解を知らないままである」ことである。この三つがそろうことで、zkSNARK として期待する安全性に近づく。

■悪い証明とよい証明

証明者が $x = 2$ をそのまま送れば、検証者は

$$2^3 + 2 + 5 = 15$$

を計算して正しさを確認できる。この方法では、検証者に秘密が渡ってしまう。

一方、zkSNARK では、証明者は x を送らず、proof π を送る。検証者は

$$\text{Verify}(y, \pi) = 1$$

だけを確認する。このとき proof system の安全性により、「有効な π を作れたなら、証明者は何らかの x を知っているはずだ」と考える。

2.7 離散対数の知識証明

ゼロ知識証明の基本例として、離散対数の知識証明を確認する。公開情報として、巡回群の生成元 g と

$$y = g^x$$

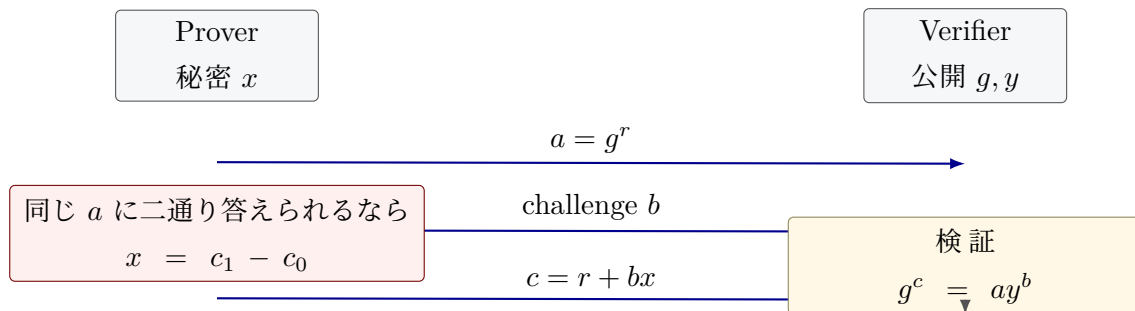
がある。証明者は秘密の x を知っていることを示したいが、 x 自体は明かしたくない。対話式のプロトコルは次の形になる。

1. 証明者は乱数 r を選び、 $a := g^r$ を検証者へ送る。
2. 検証者はチャレンジ $b \in \{0, 1\}$ をランダムに選んで送る。
3. 証明者は $c := r + bx$ を返す。
4. 検証者は

$$g^c = ay^b$$

が成り立つかを確認する。

離散対数の知識証明: 三つのメッセージ



完全性はすぐに確認できる。証明者が本当に x を知っていれば、

$$g^c = g^{r+bx} = g^r (g^x)^b = ay^b$$

だからである。

知識健全性の直感は、「証明者が $b = 0$ と $b = 1$ の両方に答えられるなら x を取り出せる」ことである。同じ a に対して

$$c_0 = r, \quad c_1 = r + x$$

の両方を答えられたなら、

$$x = c_1 - c_0$$

とわかる。つまり、どちらのチャレンジにも対応できる証明者は、実質的に x を知っている。

ゼロ知識性の直感は、検証者が見る値 (a, b, c) が、 x を知らなくても作れる形をしていることである。例えば b と c を先にランダムに選び、

$$a := g^c y^{-b}$$

と置くと、検証式 $g^c = ay^b$ は必ず成り立つ。このような「本物と区別できない会話を、秘密なしで作れる」という見方が、zero-knowledge の形式化につながる。

■注意

上のプロトコルは対話式であり、チャレンジ b は検証者が後からランダムに選ぶ必要がある。非対話化するとき、Fiat-Shamir 変換により b をハッシュ値から作る。SNARK でも、対話をハッシュで置き換える発想がよく使われる。

■確認問題

1. witness と statement の違いを、自分の言葉で説明せよ。
2. zkSNARK で「短い証明」が重要になる場面を一つ挙げよ。
3. zero-knowledge で隠したい情報と、検証者に伝えたい情報を分けて書け。

■まず覚えること

- witness は主張を成り立たせる補助入力で、zk では通常それを非公開に保つ。
- zk は「正しさ」と「秘密を漏らさないこと」を同時に扱う。
- SNARK は、短く非対話な argument of knowledge を作り、検証を速くするための仕組みである。

3 有限体、多項式、FFT

■この章で答える問い

- なぜ zk では有限体上の計算を使うのか。
- 有限体での足し算、掛け算、割り算は普通の算数と何が違うのか。
- 多項式と FFT は、証明システムのどこで出てくるのか。

3.1 有限体の直感

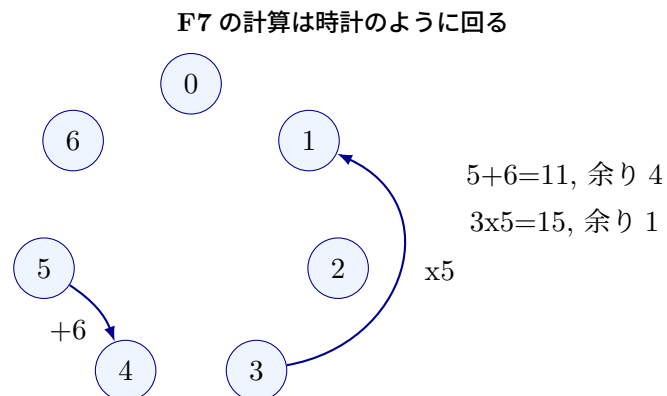
有限体は、要素数が有限個しかないにもかかわらず、足し算、引き算、掛け算、割り算がうまく定義できる世界である。最も基本的な例は、素数 p に対する剰余の世界 $\mathbb{F}_p$ である。 $\mathbb{F}_7$ なら、要素は次の 7 個だけである。

$$\mathbb{F}_7 = \{0, 1, 2, 3, 4, 5, 6\}$$

計算結果が 7 以上になったら、7 で割った余りに戻す。例えば $\mathbb{F}_7$ では、

$$5 + 6 = 11 \equiv 4 \pmod{7}, \quad 3 \cdot 5 = 15 \equiv 1 \pmod{7}$$

となる。



一般に、整数を m で割った余りの集合は $\mathbb{Z}/m\mathbb{Z}$ と書く。足し算、引き算、掛け算は $\mathbb{Z}/m\mathbb{Z}$ の中で閉じているので、これは剰余環である。しかし、割り算までできるとは限らない。素数 p のときだけ、 $\mathbb{Z}/p\mathbb{Z}$ は体になり、この体を $\mathbb{F}_p$ と書く。また、

$$\mathbb{F}_p^* := \mathbb{F}_p \setminus \{0\}$$

は 0 を除いた集合で、掛け算について巡回群になる。

3.2 体とは何か

「体」という言葉は少し大げさに見えるが、大学初学年の線形代数で出てくる実数 $\mathbb{R}$ や有理数 $\mathbb{Q}$ と同じように、足し算、引き算、掛け算、割り算が自然にできる集合だと思えばよい。有限体 $\mathbb{F}_p$ では、次の性質が成り立つ。

- 足し算と掛け算の結果が、必ず $\mathbb{F}_p$ の中に戻る。
- 足し算には単位元 0、掛け算には単位元 1 がある。
- 各要素には足し算の逆元がある。例えば $\mathbb{F}_7$ では $3 + 4 = 0$ なので、 $-3 = 4$ 。
- 0 以外の各要素には掛け算の逆元がある。

ここで p が素数であることが重要である。例えば $\mathbb{Z}/6\mathbb{Z}$ 、つまり 6 で割った余りの世界では、

$$2 \cdot 3 \equiv 0 \pmod{6}$$

となる。0 以外の二つの数を掛けた結果が 0 になってしまう。このような世界では、2 や 3 に逆元が存在しない。 $\mathbb{Z}/6\mathbb{Z}$ は、体の条件を満たさない例である。zk でよく使う $\mathbb{F}_p$ では、 p を大きな素数にすることで、割り算を含む代数計算を安定して扱える。

3.3 逆元と割り算

有限体で割り算をするには、逆元を使う。 $a \neq 0$ に対して、 $a \cdot a^{-1} = 1$ となる a^{-1} が存在する。

a	1	2	3	4	5	6
$a^{-1} \text{ in } \mathbb{F}_7$	1	4	5	2	3	6

例えば $2^{-1} = 4$ である。なぜなら $2 \cdot 4 = 8 \equiv 1 \pmod{7}$ だからである。

逆元は、方程式を解くときにも使う。例えば $\mathbb{F}_7$ で

$$3x = 5$$

を解きたいとする。普通の算数なら両辺を 3 で割る。有限体では、 $3^{-1} = 5$ なので、両辺に 5 を掛ける。

$$x = 5 \cdot 5 = 25 \equiv 4 \pmod{7}$$

実際に $3 \cdot 4 = 12 \equiv 5 \pmod{7}$ なので、答えは $x = 4$ である。

3.4 拡張 Euclid 互除法で逆元を求める

有限体の逆元は、拡張 Euclid 互除法で具体的に求められる。考え方は単純である。素数 p と $0 < x < p$ に対して、 x と p は互いに素なので、

$$\gcd(x, p) = 1$$

である。拡張 Euclid 互除法を使うと、整数 a, b で

$$ax + bp = 1$$

を満たすものを具体的に見つけられる。この式を $\text{mod } p$ で見ると、 $bp \equiv 0 \pmod{p}$ なので、

$$ax \equiv 1 \pmod{p}$$

となる。したがって、 a が $\mathbb{F}_p$ における x の逆元である。

■小例: $\mathbb{F}_{13}$ で 2^{-1} を求める

$$13 = 6 \cdot 2 + 1$$

なので、

$$1 = 13 - 6 \cdot 2$$

である。両辺を $\text{mod } 13$ で見ると、

$$-6 \cdot 2 \equiv 1 \pmod{13}$$

となる。 $-6 \equiv 7 \pmod{13}$ だから、

$$2^{-1} = 7 \in \mathbb{F}_{13}$$

である。実際、 $2 \cdot 7 = 14 \equiv 1 \pmod{13}$ である。

■注意

0 には逆元がない。これは普通の算数で 0 割りができないことと同じである。有限体を使っても、0 で割る操作は定義できない。

3.5 なぜ有限体を使うのか

zkSNARK では、計算を制約として扱う。制約は足し算と掛け算を組み合わせで表されることが多い。有限体を使うと、計算結果が常に決まった有限集合の中に収まり、代数的な議論がしやすくなる。

実数や浮動小数点数は、丸め誤差や連続性の扱いが難しい。一方、有限体ではすべての計算が厳密である。そのため、プログラムの実行や回路の制約を数学的な対象として扱いやすい。

3.6 有限体上で制約を書く

zk の arithmetization では、プログラムの条件を有限体上の等式として書く。等式は、左辺と右辺の差が 0 である、という形にできる。例えば、

$$a + b = c$$

は

$$a + b - c = 0$$

と書ける。掛け算を含む条件も同じである。

$$a \cdot b = d \iff ab - d = 0$$

boolean 値も有限体上の制約として書ける。 b が 0 または 1 であることは、

$$b(b-1) = 0$$

で表せる。なぜなら、体では積が 0 になるなら少なくとも片方が 0 なので、 $b = 0$ または $b-1 = 0$ 、つまり $b = 1$ しかないからである。

■小例: AND を制約にする

boolean 値 a, b, c に対して、 $c = a \wedge b$ を表したい。0/1 の世界では、AND は掛け算で表せる。

$$c = ab$$

したがって、有限体上の制約としては

$$ab - c = 0, \quad a(a-1) = 0, \quad b(b-1) = 0, \quad c(c-1) = 0$$

を置けばよい。最後の三つは、 a, b, c が本当に boolean 値であることを保証するための制約である。

■注意

有限体では、値は素数 p で割った余りとして扱われる。そのため、普通の整数としての大小比較やオーバーフローの感覚はそのまま使えない。例えば p に近い値も、有限体の中では一つの要素として扱われる。「大きい整数」として扱いたい場合は、そのための表現と制約を用意する。range check や大小比較には追加の制約が必要である。

3.7 巡回群と位数

有限体の非ゼロ要素は、掛け算について群を作る。ある要素 g を何度も掛けることで、群の要素を巡回できることがある。このような g を生成元と呼ぶ。有限体の非ゼロ要素が作る乗法群は巡回群であり、少なくとも一つの生成元を持つ。

$$g^0, g^1, g^2, \dots$$

FFT では、点の集合が「きれいに分解できる」構造を持っていることが重要になる。特に radix-2 FFT では、2 のべき乗サイズの部分群や、それに対応する root of unity が必要になる。

暗号で使う巡回群では、群の位数も重要である。位数が小さな因数に分解できると、離散対数問題が小さな問題に分解され、攻撃しやすくなることがある。暗号で使う巡回群の位数は、大きな素数、または大きな素数因子を持つことが望ましい。

3.8 DL、CDH、DDH

巡回群 $\mathbb{G} = \langle g \rangle$ を乗法記法で書く。暗号の安全性仮定では、次の三つがよく出る。

DL 問題 g と g^a から a を求める問題。

CDH 問題 g, g^a, g^b から g^{ab} を求める問題。

DDH 問題 g, g^a, g^b, T が与えられたとき、 $T = g^{ab}$ かランダムな g^c かを判定する問題。

加法群では同じ内容を次のように書く。

DL 問題 P と aP から a を求める。

CDH 問題 P, aP, bP から abP を求める。

DDH 問題 P, aP, bP, T から、 $T = abP$ かランダムな cP かを判定する。

楕円曲線暗号では加法記法を使うため、 aP という形が頻繁に出てくる。一方、有限体の乗法群や古典的な Diffie-Hellman では g^a という形で書く。記法は違うが、秘密スカラーを公開群要素に写すという構造は同じである。

3.9 多項式は制約の共通言語

多項式は、有限体上の計算をまとめて扱うための便利な道具である。例えば、

$$f(X) = 3X^2 + 2X + 5$$

という多項式は、係数 $(3, 2, 5)$ で表すことも、いくつかの点での評価値で表すこともできる。

係数表現 多項式を係数の列として持つ。

評価表現 いくつかの点 $x_0, x_1, \dots$ に対する $f(x_i)$ の値として持つ。

次数 d 以下の多項式は、異なる $d+1$ 点での値が決まれば一意に決まる。この事実は、zkSNARK で「大きな制約が正しく満たされているか」をランダムな点で確認する発想につながる。

■小例: 2 次多項式は 3 点で決まる

2 次以下の多項式 $f(X) = aX^2 + bX + c$ には、未知数が 3 つある。異なる 3 点での値がわかれば、原則として a, b, c を決められる。したがって、2 次以下という制限があるなら、3 点の情報で多項式全体を固定できる。

3.10 補間を式で見る

異なる点 $x_0, \dots, x_d$ と値 $y_0, \dots, y_d$ が与えられたとき、次数 d 以下の多項式 f で

$$f(x_i) = y_i \quad (i = 0, \dots, d)$$

を満たすものは一意に存在する。これを多項式補間という。具体的には、Lagrange 基底

$$L_i(X) = \prod_{\substack{0 \leq j \leq d \\ j \neq i}} \frac{X - x_j}{x_i - x_j}$$

を使うと、

$$f(X) = \sum_{i=0}^d y_i L_i(X)$$

と書ける。ここで $L_i(x_i) = 1$ 、 $L_i(x_j) = 0$ ($j \neq i$) になるように作っている。したがって、 $X = x_k$ を代入すると、和の中で $i = k$ の項だけが残り、 $f(x_k) = y_k$ になる。

zk でこの考え方が重要なのは、長い計算履歴を「点ごとの値」として持ち、それを一つの多項式にまとめられるからである。例えば実行トレースの各行の値を y_i と見れば、それらを補間してトレース多項式を作れる。

3.11 多項式が等しいことをどう確認するか

二つの多項式 f と g が等しいかを確認したいとする。すべての点で評価して比べるのは高コストである。しかし、 $h(X) = f(X) - g(X)$ がゼロ多項式でないなら、 h が 0 になる点の数は高々 $\deg(h)$ 個である。そのため、大きな集合からランダムな点 r を選んで $f(r) = g(r)$ を確認すると、嘘を見抜ける確率が高い。

この考え方は Schwartz-Zippel の補題として知られている。有限集合 S からランダムに r を選ぶとき、ゼロでない次数 d の多項式 h について、

$$\Pr_{r \leftarrow S}[h(r) = 0] \leq \frac{d}{|S|}$$

となる。つまり、評価点の候補集合が次数に比べて十分大きければ、嘘の多項式が偶然 0 になる確率は小さい。

3.12 vanishing polynomial

ある評価領域 $H = \{\omega^0, \omega^1, \dots, \omega^{n-1}\}$ 上ですべて 0 になる多項式を考える。 ω が位数 n の root of unity なら、 H の全要素は $X^n - 1$ の根である。このとき

$$Z_H(X) = X^n - 1$$

を vanishing polynomial と呼ぶ。制約が H 上のすべての点で成り立つことは、制約違反を表す多項式 $C(X)$ が $Z_H(X)$ で割り切れることとして表せる。

$$C(\omega^i) = 0 \text{ for all } i \iff Z_H(X) \mid C(X)$$

この「たくさんの点で 0」を「ある多項式で割り切れる」に変える発想が、Plonk や STARK など で頻繁に現れる。

■注意

この説明は直感であり、実際の proof system ではランダム性、commitment、Fiat-Shamir 変換などを組み合わせて安全性を設計する。1 点検査を安全に使うには、評価点の選び方、多項式の次数、攻撃者の能力を合わせて管理する。

3.13 FFT は何を高速化するか

FFT は、係数表現と評価表現の変換を高速化する技法である。素朴に n 個の点で n 次未満の多項式を評価すると、おおよそ $O(n^2)$ の計算が必要になる。FFT を使える点集合を選ぶと、これを $O(n \log n)$ にできる。

有限体上の FFT も、複素数上の FFT と同じ形をしている。係数

$$f(X) = a_0 + a_1X + \cdots + a_{n-1}X^{n-1}$$

を持つ多項式を、評価点

$$1, \omega, \omega^2, \dots, \omega^{n-1}$$

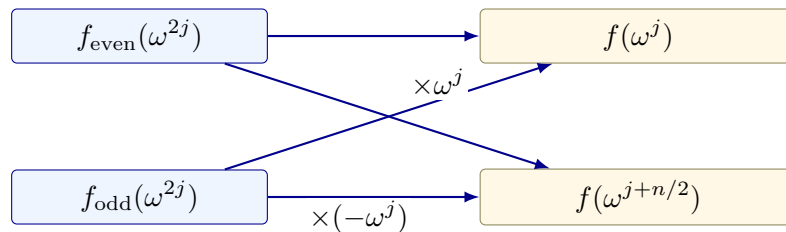
で評価する。ここで $\omega^n = 1$ かつ $0 < k < n$ では $\omega^k \neq 1$ となる ω を、位数 n の root of unity と呼ぶ。評価値は

$$\hat{a}_j = f(\omega^j) = \sum_{i=0}^{n-1} a_i \omega^{ij}$$

である。この式をそのまま計算すると、各 j について n 個の項を足すので $O(n^2)$ になる。FFT では、多項式を偶数次と奇数次に分ける。

$$f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2)$$

FFT の蝶形: 偶数次と奇数次を合流させる



すると、

$$f(\omega^j) = f_{\text{even}}(\omega^{2j}) + \omega^j f_{\text{odd}}(\omega^{2j})$$

となる。 ω^{2j} はサイズ $n/2$ の評価点集合を巡るため、問題を半分のサイズに分解できる。この分解を再帰的に行うことで $O(n \log n)$ の計算量になる。

zk では、大きな制約列、実行トレース、selector、wire 値などを多項式として扱う。そのため、係数表現と評価表現を高速に行き来できることが、証明生成の性能に大きく影響する。

方法	計算量の直感	使う場面
ナイーブ評価	各点で多項式を一から計算する	小さい例、教材、検証用実装
FFT	点集合の対称性を使って再帰的に分解する	大規模な証明生成、多点評価、補間

■確認問題

1. $\mathbb{F}_7$ で $6 + 5$ 、 $4 \cdot 5$ 、 3^{-1} を計算せよ。
2. 次数 d の多項式を一意に決めるには、何点の値が必要か。
3. FFT が高速化している変換を二つの表現名で答えよ。

■まず覚えること

- 有限体は、割り算を含む代数計算を有限集合の中で完結させる舞台である。
- 多項式は、制約や計算履歴をまとめて扱うための共通言語である。
- FFT は、多項式の係数表現と評価表現を高速に変換する技法である。

4 楕円曲線

■この章で答える問い

- 楕円曲線はなぜ commitment や SNARK で使われるのか。
- 離散対数問題とは何か。
- ペアリングは何を可能にするのか。

4.1 点を足せる世界

楕円曲線暗号では、曲線の形を暗記するより、曲線上の点に対して次の操作ができることが重要である。

- 点 P と点 Q を足して、別の点 $P + Q$ を得る。
- 点 P を整数 x 回足して、スカラー倍 xP を得る。

スカラー倍 xP は効率よく計算できる。一方で、 P と xP から x を求めることは難しいと考えられている。これが離散対数問題の直感である。

4.2 楕円曲線を式で見る

暗号でよく使う楕円曲線は、有限体 $\mathbb{F}_p$ 上の点集合として定義される。典型的には、

$$E/k : y^2 = x^3 + ax + b$$

という形の方程式を満たす点 (x, y) を集める。ここで k は体であり、暗号では多くの場合 $k = \mathbb{F}_p$

またはその拡大体である。滑らかな曲線として扱うために

$$4a^3 + 27b^2 \neq 0$$

という条件を置く。この短い Weierstrass 形は、標数が 2 でも 3 でもない体で使う標準的な形である。有限体上では、 x や y は $\mathbb{F}_p$ の要素であり、すべての計算は $\text{mod } p$ で行う。

楕円曲線上の点には、特別な点 $\mathcal{O}$ を追加する。 $\mathcal{O}$ は足し算の単位元であり、

$$P + \mathcal{O} = P$$

を満たす。この $\mathcal{O}$ を「無限遠点」と呼ぶ。

記法としては、

$$E(k) = \{(x, y) \in k^2 \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

と書く。これは、体 k の元を座標にもつ楕円曲線上の点全体である。ペアリングや実装では、点の座標が拡大体 $\mathbb{F}_{p^n}$ に入ることもある。

4.3 点の足し算の式

二つの点 $P = (x_1, y_1)$ 、 $Q = (x_2, y_2)$ を足すと、また曲線上の点 $R = P + Q$ が得られる。 $P \neq Q$ の場合、傾き

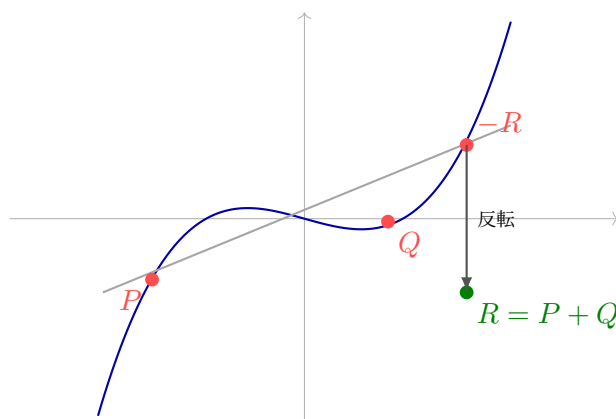
$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

を使って、

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

と計算する。この (x_3, y_3) が $P + Q$ である。割り算は有限体での逆元を使う。

楕円曲線の点加算：直線で交わり、上下を反転する



$P = Q$ の場合、つまり点を 2 倍する場合は接線の傾きを使う。

$$\lambda = \frac{3x_1^2 + a}{2y_1}$$

として同じ形の式で $2P$ を計算する。この「足せる」構造があるから、楕円曲線上で $xP = P + \dots + P$ というスカラー倍を定義できる。

4.4 ねじれ点 $E[m]$

自然数 m に対して、

$$E[m] = \{P \in E \mid mP = \mathcal{O}\}$$

を m 等分点、または m -torsion points、ねじれ点の集合と呼ぶ。ペアリングを扱うときにも中心的に使われる記法である。直感的には、 m 回足すと単位元に戻る点の集合である。

ペアリングの数学的定義では、Weil ペアリングのように

$$e : E[m] \times E[m] \rightarrow k^*$$

という形の写像が出てくる。実用的な pairing-friendly curve では、効率や安全性の都合から G_1 、 G_2 、 G_T という部分群に整理して扱うことが多い。

■注意

上の式は概念理解のための標準形である。実際の曲線実装では、無限遠点、分母が 0 になる場合、座標系、subgroup check などを厳密に扱う必要がある。

4.5 スカラー倍はなぜ速いか

xP は、 P を x 回足すという意味である。実装では、整数の二進展開を使って、倍々にしながら必要な点だけ足す。

例えば $13P$ は、

$$13P = 8P + 4P + P$$

である。まず $2P$ 、 $4P$ 、 $8P$ を doubling で作り、最後に必要なものを足す。この方法なら、計算回数は x に比例せず、だいたい $\log x$ に比例する。だから大きなスカラーでも xP は効率よく計算できる。

4.6 離散対数問題

楕円曲線上の離散対数問題は、次の形で現れる。

$$\text{Given } P \text{ and } Q = xP, \text{ find } x.$$

順方向の計算 xP は速い。逆方向に x を探すのは難しい。この非対称性が、署名、Pedersen commitment、さまざまな proof system の安全性の土台になる。

この問題は、有限体の乗法群での離散対数問題と似ている。有限体なら $g^x = h$ から x を探す問題であり、楕円曲線なら $xP = Q$ から x を探す問題である。どちらも「前に進む計算は速いが、戻す計算は難しい」という片方向性を利用している。

■Pedersen commitment への接続

Pedersen commitment は、典型的には

$$\text{Com}(m; r) = mG + rH$$

のような形を持つ。ここで m は commit したい値、 r は hiding のための乱数、 G, H は楕円曲線上の基点である。値 m を隠す主役は乱数 r であり、binding には G と H の間の離散対数関係が知られていないことが効いている。

4.7 ペアリング

ペアリングは、二つの群の要素を別の群へ写す特殊な演算である。詳細な構成は後回しにして、ここでは記法を丁寧に分ける。数学的な説明では、まず楕円曲線のねじれ点を使って

$$e : E[m] \times E[m] \rightarrow k^*$$

のような対称ペアリングを導入している。一方、KZG などの実用的なプロトコルでは、しばしば

$$e : G_1 \times G_2 \rightarrow G_T$$

という非対称ペアリングとして扱う。ここで G_1 と G_2 は楕円曲線上の加法群、 G_T は有限体の拡大体などに入る乗法群である。

ペアリングで一番重要なのは、次の双線形性である。

$$e(aP, bQ) = e(P, Q)^{ab}$$

KZG commitment では、ペアリングを使って「commit された多項式が、ある点である値を取る」という関係を短く検証する。つまり、ペアリングは多項式 commitment の短い opening proof を支える重要な道具である。

双線形性を分けて書くと、

$$e(aP, Q) = e(P, Q)^a, \quad e(P, bQ) = e(P, Q)^b$$

したがって、

$$e(aP, bQ) = e(P, Q)^{ab}$$

となる。この性質により、「左側の群でスカラー倍された値」と「右側の群でスカラー倍された値」の関係を、 G_T の中で比較できる。KZG では、commitment の中に隠れている $f(\tau)$ と、proof の中に隠れている商多項式の値 $q(\tau)$ の関係を、この双線形性で検証する。

もう一つ重要な条件は、非退化性である。これは、生成元の情報が G_T 側にも残るという条件である。暗号では、生成元 $P \in G_1$ 、 $Q \in G_2$ に対して $e(P, Q)$ が G_T の十分大きな位数の元になるような群を選ぶ。

■注意

Weil ペアリングのような対称的な説明では $e(P, P) = 1$ となる性質が出ることもある。一方、KZG で使う記法では $P \in G_1$ 、 $Q \in G_2$ を別の群の生成元として取り、 $e(P, Q)$ が非自明な値になるように使う。同じ「ペアリング」という言葉でも、入力群と使い方を確認することが大切である。

■注意

楕円曲線やペアリングは、実装上の罠が多い領域である。曲線、群の位数、cofactor、subgroup check、serialization の扱いを軽視すると、深刻な脆弱性につながる。教材内では直感を優先するが、実装では必ず信頼できるライブラリと仕様に従う。

■確認問題

1. P と xP から x を求める問題を何と呼ぶか。
2. Pedersen commitment で乱数 r を入れる理由を説明せよ。
3. KZG でペアリングが果たす役割を一文で書け。

■まず覚えること

- 楕円曲線は、簡単に進めるが戻りにくい計算を作るために使われる。
- 離散対数問題は、 P と xP から x を求めることの難しさである。
- ペアリングは、楕円曲線上の隠れた関係を検証するための道具である。

5 Commitment Scheme

■この章で答える問い

- commitment とは何か。
- binding と hiding はどのような性質か。
- Merkle tree、Pedersen commitment、KZG commitment は何が違い、どこで使い分けるのか。

5.1 commitment の基本

commitment scheme は、「今決めた値を固定し、あとで開示できるが、途中で別の値に変えられない」ための仕組みである。封筒に値を入れて封をする比喻で考えるとわかりやすい。

commit 値を固定し、commitment を作る。

open 後で値と補助情報を公開する。

verify commitment と公開された値が対応しているか確認する。

抽象的には、commitment scheme は次の三つのアルゴリズムで書ける。

$$c \leftarrow \text{Com}(m; r)$$

$$o \leftarrow \text{Open}(m; r)$$

$$\text{Verify}(c, m, o) \in \{0, 1\}$$

ここで m は commit したい message、 r は乱数、 c は commitment、 o は opening information である。ランダム性 r がある場合、同じ m でも異なる c を作れる。これが hiding に効く。一方で、binding は「一つの c を、二つの違う message として開けない」ことを要求する。

$$\text{Verify}(c, m, o) = 1 \text{ and } \text{Verify}(c, m', o') = 1 \Rightarrow m = m'$$

実際には、この含意は「効率的な攻撃者には破れない」という計算量的な意味で述べられることが多い。

重要な性質は二つある。ただし、実際の構成によってどちらをどの強さで満たすかは異なる。例えば、Merkle root はデータ集合への binding にはよく使われるが、データが小さい場合や推測可能な場合には、それだけで hiding を提供するとは限らない。

Binding commit した後で、別の値に開けないこと。

Hiding open するまで、commitment から値が漏れないこと。

■発展内容

commitment scheme は、より厳密には次のアルゴリズムの組として定義できる。

$$pp \leftarrow \text{Setup}(1^\lambda)$$

$$(c, d) \leftarrow \text{Commit}(pp, m)$$

$$b \leftarrow \text{Verify}(pp, c, m, d)$$

ここで pp は公開パラメータ、 c は commitment、 d は decommitment または opening information、 $b \in \{0, 1\}$ は検証結果である。公開パラメータを使わない単純な構成では、 pp を省略してよい。

Correctness 正直に commit して open した値は受理される。任意の message m について、

$$\Pr[\text{Verify}(pp, c, m, d) = 1 \mid pp \leftarrow \text{Setup}(1^\lambda), (c, d) \leftarrow \text{Commit}(pp, m)] \geq 1 - \text{negl}(\lambda)$$

が成り立つ。

Binding 効率的な攻撃者が、一つの commitment を二つの異なる message として open する確率は無視できる。任意の効率的な攻撃者 A について、

$$\Pr \left[\begin{array}{l} m \neq m', \\ \text{Verify}(pp, c, m, d) = 1, \\ \text{Verify}(pp, c, m', d') = 1 \end{array} \middle| \begin{array}{l} pp \leftarrow \text{Setup}(1^\lambda), \\ (c, m, d, m', d') \leftarrow A(pp) \end{array} \right] \leq \text{negl}(\lambda)$$

を要求する。

Hiding 攻撃者が二つの message m_0, m_1 を選んでも、commitment がどちらから作られたかを当てにくい。実験では $b \leftarrow \{0, 1\}$ をランダムに選び、

$$(c, d) \leftarrow \text{Commit}(pp, m_b)$$

として c だけを攻撃者へ渡す。攻撃者の出力 b' について、

$$\left| \Pr[b' = b] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

なら、computational hiding を満たす。

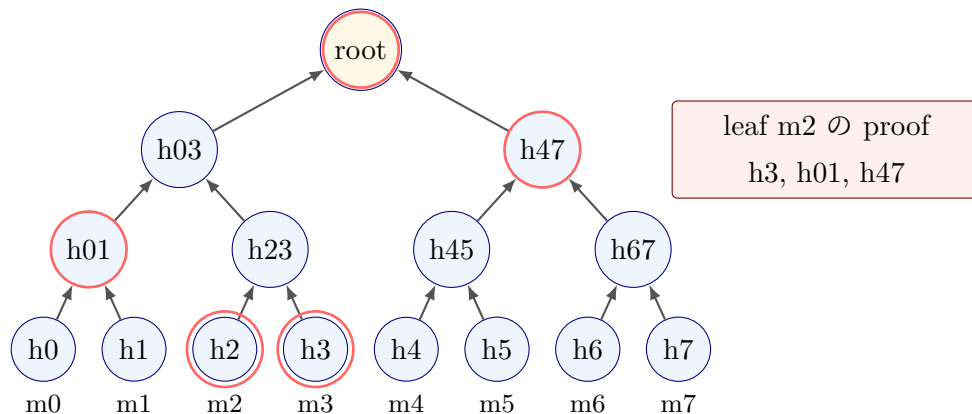
binding と hiding には、perfect、statistical、computational といった強さの違いがある。Merkle tree、Pedersen commitment、KZG commitment は、何を仮定してどの性質を満たすかが異なるため、用途に応じて使い分ける。

5.2 Merkle tree

Merkle tree は、大量のデータに対する commitment として使われる。leaf にデータのハッシュを置き、隣り合うノードをハッシュして親を作り、最終的な root を commitment として扱う。

ある leaf が木に含まれることを示すには、root までの経路に必要な sibling hash だけを示せばよい。証明サイズは $O(\log n)$ であり、構造が単純で実装しやすい。安全性は主にハッシュ関数の衝突困難性や第二原像困難性に依存する。秘匿性が必要な場合は、leaf に十分なランダム性を混ぜるなど、別の設計が必要になる。

Merkle proof: 必要なのは root までの sibling だけ



ハッシュ関数に求められる性質は、目的によって少し違う。ハッシュ関数の安全性を考えると、少なくとも次を区別しておくといよい。

一方向性 $h = \text{Hash}(m)$ から元の m を見つけるのが難しいこと。

第二原像困難性 与えられた m に対して、同じ hash になる別の $m' \neq m$ を見つけるのが難しいこと。

衝突困難性 任意の異なる m, m' で $\text{Hash}(m) = \text{Hash}(m')$ となる組を見つけるのが難しいこと。

Merkle tree の binding には、特に衝突困難性や第二原像困難性が効く。もし衝突を作れるなら、同じ root に対して別の leaf 集合を開ける可能性がある。

5.3 Merkle proof を式で追う

leaf の値を $m_0, \dots, m_7$ とする。まず leaf hash を

$$h_i = \text{Hash}(m_i)$$

と作る。次に隣り合う hash を結合して親を作る。

$$h_{0,1} = \text{Hash}(h_0 \| h_1), \quad h_{2,3} = \text{Hash}(h_2 \| h_3)$$

同じことを上に向かって繰り返し、最後の root を得る。

$$r = \text{Hash}(h_{0,1,2,3} \| h_{4,5,6,7})$$

例えば m_2 が含まれることを示したいなら、検証者に必要なのは次の情報である。

$$m_2, \quad h_3, \quad h_{0,1}, \quad h_{4,5,6,7}$$

検証者は

$$h_2 = \text{Hash}(m_2)$$

から始めて、

$$h_{2,3} = \text{Hash}(h_2 \| h_3)$$

を計算し、さらに $h_{0,1}$ と結合して $h_{0,1,2,3}$ を作り、最後に root を再計算する。再計算した root が公開された r と一致すれば、 m_2 が木に含まれると確認できる。このとき、検証者は経路上の情報だけを読む。

■注意

左右の順序は重要である。 $\text{Hash}(h_2 \| h_3)$ と $\text{Hash}(h_3 \| h_2)$ は一般に別の値になる。実装では、各 sibling が左側か右側かを proof に含める必要がある。

■Merkle proof の直感

8 個の leaf を持つ木では、ある leaf から root までに 3 段の経路がある。検証者は、leaf の値と 3 個の sibling hash を使って root を再計算する。再計算した root が公開 root と一致すれば、その leaf が木に含まれると確認できる。

5.4 Pedersen commitment

Pedersen commitment は、楕円曲線のスカラー倍を使う commitment である。典型的には次の形を持つ。

$$\text{Com}(m; r) = mG + rH$$

m は値、 r は乱数、 G, H は基点である。乱数 r によって強い hiding を実現し、 G と H の離散対数関係を誰も知らないという前提と離散対数問題の難しさによって binding を支える。

Pedersen commitment の大きな特徴は、加法準同型性である。

$$\text{Com}(a; r_a) + \text{Com}(b; r_b) = \text{Com}(a + b; r_a + r_b)$$

この性質により、commit された値を開かずに、和に関する関係を扱える。

5.5 Pedersen の hiding と binding を式で見る

Pedersen commitment の hiding は、乱数 r が commitment をずらすことで生まれる。同じ m でも、 r が違えば

$$mG + rH$$

は別の点になる。検証者が commitment C だけを見ても、どの m と r の組から来たのかを区別できない。直感的には、 rH が十分にランダムなマスクとして働く。

binding は、同じ commitment を二通りに開けると何が起こるかを見ると理解しやすい。もし

$$mG + rH = m'G + r'H$$

かつ $m \neq m'$ なら、移項して

$$(m - m')G = (r' - r)H$$

となる。仮に $H = \alpha G$ という関係の α がわかってしまうと、この式を使って別の開き方を作れる可能性がある。だから、Pedersen commitment では G と H の間の離散対数関係が誰にも知られていないことが重要である。

■注意

加法準同型は便利だが、値は群の位数を法として扱われる。例えば位数が q の群では、 q と 0 は同じスカラーとして扱われる。整数としての範囲を保証したい場合は、range proof や追加制約が必要である。

5.6 KZG commitment

KZG commitment は、Kate、Zaverucha、Goldberg による多項式 commitment である。多項式 $f(X)$ を短い commitment に固定し、後で「 $f(z) = y$ である」ことを短い proof で示せる。

KZG の特徴は次の通りである。

- 多項式の opening proof が定数サイズになる。
- 検証にペアリングを使う。
- Structured Reference String、または SRS と呼ばれる公開パラメータを使う。
- 多くの実用構成では、SRS 生成時の秘密、いわゆる toxic waste が破棄されていることを前提にする。
- Ethereum blob や Verkle tree の文脈で重要になる。

■注意

KZG の trusted setup や SRS は、検証の信頼を支える公開パラメータである。setup の秘密が漏れたり、不正に生成されたりすると、主に binding が壊れ、不正な opening proof を作れる可能性がある。過去に commit されたデータを直接復号する話とは別に、検証の信頼性へ深刻に影響する。実用では、どの setup を信頼しているのかを明確にする必要がある。

基本的な KZG は、多項式評価を短く証明するための commitment として理解するとよい。zero-knowledge 性や多項式そのものの秘匿が必要な場合は、ランダム化や hiding 版の構成を別途確認する。

KZG の binding は、直感的には「 τ を知らないまま、 τ に関する都合のよい関係を偽造できない」ことに依存する。ペアリングを使う安全性仮定には、 n -SDH、BDH、DBDH などがある。KZG でよく出る SDH 系仮定は、その系統の仮定である。細部はプロトコルごとに異なるが、初学者向けには「SRS に含まれる $P, \tau P, \dots, \tau^d P$ から、未知の τ を利用した不正な opening を作るのが難しい」という理解で十分である。

5.7 KZG opening を式で見る

KZG では、setup で秘密の値 τ を選び、左側の群 G_1 の生成元を P 、右側の群 G_2 の生成元を Q として、次のような SRS を公開する。

$$P, \tau P, \tau^2 P, \dots, \tau^d P$$

検証用には、少なくとも Q と τQ も使う。実際には τ そのものは破棄され、公開されるのはこれらの群要素だけである。多項式

$$f(X) = a_0 + a_1 X + \dots + a_d X^d$$

への commitment は、

$$C = a_0P + a_1(\tau P) + \cdots + a_d(\tau^d P)$$

である。これは形式的には

$$C = f(\tau)P$$

と見なせる。ただし、証明者も検証者も τ の値は知らない。

次に、ある点 z での値 $y = f(z)$ を open したいとする。多項式の因数定理より、 $f(z) = y$ であることは

$$f(X) - y$$

が $X - z$ で割り切れることと同値である。そこで商多項式

$$q(X) = \frac{f(X) - y}{X - z}$$

を作る。KZG proof は、この商多項式への commitment

$$\pi = q(\tau)P$$

である。

検証では、ペアリングを使って

$$e(C - yP, Q) = e(\pi, \tau Q - zQ)$$

を確認する。左辺は

$$e((f(\tau) - y)P, Q)$$

であり、右辺は

$$e(q(\tau)P, (\tau - z)Q)$$

である。双線形性により、右辺は $q(\tau)(\tau - z)$ を指数に持つ値になる。したがって、両辺が一致することは

$$f(\tau) - y = q(\tau)(\tau - z)$$

という関係を確認していると読める。これは $q(X)(X - z) = f(X) - y$ という多項式関係を、秘密の点 τ で検査している、という直感で理解できる。

5.8 比較表

Scheme	対象	主な仮定	証明サイズ	強み	注意点
Merkle tree	配列、木	ハッシュ関数の衝突困難性など	$O(\log n)$	単純、透明、実装しやすい	hiding は別途設計が必要
Pedersen	値、ベクトル	離散対数問題、基点間の未知関係	用途次第	強い hiding、加法準同型	乱数再利用や基点生成に注意
KZG	多項式	ペアリング、SDH 系仮定、SRS	$O(1)$	短い proof、高速検証	SRS 前提、対応曲線が必要

5.9 zkSNARK での役割

証明システムでは、制約、実行トレース、多項式、wire 値など、大きな対象が頻繁に現れる。検証者がそれらを全部読むなら、検証は遅くなる。commitment scheme は、大きな対象を短い値で代表させ、必要な点だけを open するために使われる。

■確認問題

1. binding と hiding の違いを説明せよ。
2. Merkle tree の証明サイズが $O(\log n)$ になる理由を直感的に説明せよ。
3. Pedersen commitment の加法準同型性が役立つ場面を一つ挙げよ。
4. KZG が多項式 commitment として便利な理由を一つ挙げよ。

■まず覚えること

- commitment は「今決めたことを後で開示できるが、途中で変えられない」仕組みである。
- Merkle はデータ構造、Pedersen は準同型、KZG は多項式 commitment として覚える。
- 証明システムでは、commitment が大きな対象を短い値で代表させる。

6 Arithmetization

■この章で答える問い

- プログラムや計算を、どうやって証明可能な形に変換するのか。
- R1CS、AIR、Plonkish arithmetization、CCS は何が違うのか。
- arithmetization の違いは、開発体験や証明システムにどう影響するのか。

6.1 arithmetization の目的

arithmetization は、「計算が正しく実行された」という主張を、有限体上の制約として表す工程である。普通のプログラムには、分岐、ループ、メモリ、型、関数呼び出しなどがある。証明システムは、それらを足し算と掛け算を中心とした代数的な制約として扱う。

この変換は、コンパイラに似ている。開発者が書く高レベルなコードや DSL は、内部で低レベルな制約へ展開される。したがって、zk アプリケーション開発では「自分が書いたコードがどのような制約になるか」を意識することが重要である。

6.2 プログラムを制約にするとはどういうことか

次のような短いプログラムを考える。

$$\begin{aligned}t_1 &\leftarrow x \cdot x \\t_2 &\leftarrow t_1 \cdot x \\y &\leftarrow t_2 + x + 5\end{aligned}$$

普通のプログラムでは、上から順に値を計算する。arithmetization では、この実行が正しかったことを、変数たちが満たすべき等式として書く。

$$t_1 - x^2 = 0, \quad t_2 - t_1 x = 0, \quad y - t_2 - x - 5 = 0$$

証明者は、 x, t_1, t_2 の値を witness として持つ。検証者は、公開値 y と proof だけを見て、これらの等式を満たす値が存在することを確認したい。つまり、プログラムの実行を「存在する変数の組が制約を満たす」という数学の問題に変えている。

■小例: witness assignment

$x = 2$ のとき、

$$t_1 = 4, \quad t_2 = 8, \quad y = 15$$

である。制約に代入すると、

$$4 - 2^2 = 0, \quad 8 - 4 \cdot 2 = 0, \quad 15 - 8 - 2 - 5 = 0$$

となり、すべて満たされる。

もし $y = 16$ と主張したら、最後の式は

$$16 - 8 - 2 - 5 = 1$$

となり、制約違反が検出される。

6.3 R1CS

R1CS は、Rank-1 Constraint System の略である。各制約は、直感的には次の形をしている。

$$A \cdot B = C$$

ここで A, B, C は変数の線形結合である。掛け算を一つずつ制約として分解していくため、算術回路を表すのに向いている。

より正確には、変数ベクトル

$$z = (1, \text{public inputs}, \text{witness variables})$$

を用意し、各制約を

$$\langle A_i, z \rangle \cdot \langle B_i, z \rangle = \langle C_i, z \rangle$$

という形で書く。 $\langle A_i, z \rangle$ は、係数ベクトル A_i と変数ベクトル z の内積、つまり線形結合である。すべての i についてこの等式が成り立てば、R1CS を満たす。

■小例: $x^3 + x + 5 = y$ を R1CS 風に分解する

秘密入力を x 、公開入力を y とする。中間変数を二つ用意する。

$$t_1 = x \cdot x, \quad t_2 = t_1 \cdot x$$

最後に、

$$t_2 + x + 5 = y$$

を制約として加える。R1CS では、最後の足し算だけの等式も、必要に応じて $1 \cdot (t_2 + x + 5 - y) = 0$ のような形に整える。

■R1CS の線形結合として書く

変数ベクトルを

$$z = (1, y, x, t_1, t_2)$$

とする。制約 $t_1 = x \cdot x$ は、

$$\langle A, z \rangle = x, \quad \langle B, z \rangle = x, \quad \langle C, z \rangle = t_1$$

となるように A, B, C を選ぶ。

制約 $t_2 = t_1 \cdot x$ は、

$$\langle A, z \rangle = t_1, \quad \langle B, z \rangle = x, \quad \langle C, z \rangle = t_2$$

である。

最後の $t_2 + x + 5 = y$ は、

$$1 \cdot (t_2 + x + 5 - y) = 0$$

と書ける。このように、R1CS では「掛け算 1 個を含む等式」に合わせて計算を細かく分解する。

6.4 AIR

AIR は、Algebraic Intermediate Representation の略である。計算を実行トレースとして表し、隣り合う行の関係を遷移制約で確認する。繰り返し計算、VM、ステートマシンのような対象に向いている。

AIR では、計算を「横に変数、縦に時間」を持つ表として見る。各行はある時刻の状態であり、隣の行へ正しく遷移しているかを制約で確認する。R1CS が「回路の部品を並べる」見方に近いのに対し、AIR は「実行ログが正しいかを見る」見方に近い。

■小例: Fibonacci を AIR 風に見る

Fibonacci 列を考える。trace table に各 step の状態を並べる。

step	a	b
0	0	1
1	1	1
2	1	2
3	2	3

遷移制約は、

$$a_{i+1} = b_i, \quad b_{i+1} = a_i + b_i$$

である。boundary constraint として、最初の行や最後の行が期待値になっていることも確認する。

6.5 AIR を多項式制約にする

AIR では、trace table の各列を多項式として補間する。例えば列 a と列 b から、多項式 $A(X)$ 、 $B(X)$ を作る。評価領域を

$$H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

とし、行 i の値が

$$A(\omega^i) = a_i, \quad B(\omega^i) = b_i$$

になるようにする。Fibonacci の遷移

$$a_{i+1} = b_i, \quad b_{i+1} = a_i + b_i$$

は、多項式では

$$A(\omega X) - B(X) = 0$$

$$B(\omega X) - A(X) - B(X) = 0$$

が評価領域上で成り立つ、という条件になる。ここで $X = \omega^i$ と置くと、 $\omega X = \omega^{i+1}$ なので、次の行を参照していることがわかる。

boundary constraint は、例えば

$$A(1) = 0, \quad B(1) = 1$$

のように、最初の行の値を固定する。最後の行を固定したい場合も、対応する評価点での値を指定する。

6.6 Plonkish arithmetization

Plonkish arithmetization では、計算を長方形の表として並べる。行は「計算の場所」、列は「値の種類」と考えるとよい。各セルには有限体の元が入る。R1CS が掛け算を一つずつ制約に分解する見方だとすると、Plonkish は「各行にどの形の制約を置くか」を設計する見方である。

row	a	b	c	q_{mul}	q_{add}
0	a_0	b_0	c_0	1	0
1	a_1	b_1	c_1	0	1
2	a_2	b_2	c_2	1	0

この表で、 a, b, c は計算中の値を置く列、 $q_{\text{mul}}, q_{\text{add}}$ はどの gate を有効にするかを表す列である。例えば $q_{\text{mul}} = 1$ の行では掛け算 gate を使い、 $q_{\text{add}} = 1$ の行では足し算 gate を使う。

advice column 証明者が witness から埋める列。秘密の中間値は主にここに入る。

instance column 公開入力を置く列。検証者も値を知っている。

fixed column 回路ごとに固定された列。selector や lookup table の値を置く。

wire gate が読むセルの値。たとえば現在行の a_i, b_i, c_i など。

selector その行でどの制約を有効にするかを選ぶ固定係数。

gate 行ごとに満たすべき多項式制約。足し算、掛け算、カスタム演算などを表す。

copy constraint 別の場所にあるセルが同じ値であることを示す制約。

lookup ある値や値の組が、事前定義された表に含まれることを示す仕組み。

Plonkish で大切なのは、表の各行を直接チェックするだけで終わらず、列全体を多項式として扱う点である。行数を n 、評価領域を

$$H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

とする。列 a は、各行の値 a_i を満たす多項式 $A(X)$ として表す。

$$A(\omega^i) = a_i$$

同様に、列 $b, c, q_{\text{mul}}, q_{\text{add}}$ も多項式 $B(X), C(X), Q_{\text{mul}}(X), Q_{\text{add}}(X)$ になる。この変換により、「すべての行で gate が満たされる」という条件を、多項式が H 上でゼロになる条件として扱える。

6.7 Plonkish の gate を式で見る

Plonkish arithmetization では、各行にいくつかの wire 値を置く。単純な例として、各行に

$$a_i, \quad b_i, \quad c_i$$

という 3 本の wire があるとする。足し算 gate なら、

$$a_i + b_i - c_i = 0$$

を満たす行として使う。掛け算 gate なら、

$$a_i b_i - c_i = 0$$

を満たす行として使う。selector を使うと、同じ表の中で行ごとに有効な gate を切り替えられる。
例えば

$$q_M a_i b_i + q_L a_i + q_R b_i + q_O c_i + q_C = 0$$

という形を考える。掛け算 gate にしたい行では $q_M = 1$ 、 $q_O = -1$ 、他を 0 にする。足し算 gate にしたい行では $q_L = q_R = 1$ 、 $q_O = -1$ 、他を 0 にする。

gate	q_M	q_L	q_R	q_O	q_C	得られる制約
mul	1	0	0	-1	0	$a_i b_i - c_i = 0$
add	0	1	1	-1	0	$a_i + b_i - c_i = 0$
const	0	1	0	0	$-k$	$a_i - k = 0$

列全体で書くと、次の多項式

$$G(X) = Q_M(X)A(X)B(X) + Q_L(X)A(X) + Q_R(X)B(X) + Q_O(X)C(X) + Q_C(X)$$

が評価領域 H 上でゼロになることを要求する。つまり、

$$G(\omega^i) = 0 \quad (i = 0, \dots, n-1)$$

である。これはさらに、vanishing polynomial

$$Z_H(X) = X^n - 1$$

を使って、

$$Z_H(X) \mid G(X)$$

と表せる。この「行ごとの制約」を「割り切りの主張」に変えるところが、Plonkish を多項式 commitment と組み合わせやすくする。

■小例: $x^3 + x + 5 = y$ を Plonkish 風に配置する

秘密入力を x 、公開入力を y とする。中間値を

$$t_1 = x^2, \quad t_2 = t_1 x, \quad t_3 = x + 5$$

と置く。3本の advice 列 a, b, c を使うと、次のような配置にできる。

row	a	b	c	gate
0	x	x	t_1	$a \cdot b = c$
1	t_1	x	t_2	$a \cdot b = c$
2	x	5	t_3	$a + b = c$
3	t_2	t_3	y	$a + b = c$

この表だけを見ると、row 0 の t_1 と row 1 の t_1 が同じ値であること、各行に現れる x が同じ値であること、row 1 の t_2 と row 3 の t_2 が同じ値であること、row 2 の t_3 と row 3 の t_3 が同じ値であること、row 3 の y が公開入力 y と同じ値であることも制約として入れる必要がある。そこで copy constraint を加える。

$$c_0 = a_1, \quad a_0 = b_0 = b_1 = a_2, \quad c_1 = a_3, \quad c_2 = b_3, \quad c_3 = y_{\text{public}}$$

のように、同じ値として扱うセルを結ぶ。

copy constraint は、別々の行や列に置かれた値が同じであることを保証する。例えば、ある行で計算した t_1 を次の行でも使いたい場合、「この二つのセルは同じ値である」という制約が必要になる。Plonk 系では、この同値関係を permutation argument などまとめて扱う。

6.8 copy constraint と permutation argument

Plonkish の copy constraint は、「同じ値であるべきセルたち」を集合として指定する。例えば、次の三つのセルが同じ値を持つべきだとする。

$$(a, 0), \quad (b, 3), \quad (c, 7)$$

これは

$$a_0 = b_3 = c_7$$

という意味である。全ての copy constraint を一つずつ検証すると数が多くなる。そこで Plonk 系では、セル位置全体に permutation を作る。同じ値であるべきセルを一つの cycle に並べ、

$$(a, 0) \mapsto (b, 3) \mapsto (c, 7) \mapsto (a, 0)$$

のようにする。証明者は、値の列がこの permutation の前後で同じ多重集合に見えることを示す。

少し式で見る。各セル位置に一意的なラベルを付ける多項式を $ID_j(X)$ 、copy constraint に従って並べ替えたラベルを $\Sigma_j(X)$ とする。ここで j は列番号で、例として $j = 1, 2, 3$ が a, b, c に対応すると考える。検証者がランダムな β, γ を選ぶと、正しい割り当てでは次のような積が一致する。

$$\prod_{i=0}^{n-1} \prod_{j=1}^3 (w_j(\omega^i) + \beta ID_j(\omega^i) + \gamma) = \prod_{i=0}^{n-1} \prod_{j=1}^3 (w_j(\omega^i) + \beta \Sigma_j(\omega^i) + \gamma)$$

ここで $w_1 = A, w_2 = B, w_3 = C$ である。左辺は元のセル位置で値を見る積、右辺は copy constraint でつないだ先のセル位置で値を見る積である。同じ値として結ばれたセルが本当に同じなら、二つの積は一致する。どこかで値がずれると、ランダムな β, γ に対して偶然一致する確率は小さくなる。

実際のプロトコルでは、この積全体をそのまま計算する形よりも、grand product polynomial $Z(X)$ を使って段階的に確認する。直感的には、

$$Z(\omega^{i+1}) = Z(\omega^i) \cdot \frac{\prod_j (w_j(\omega^i) + \beta ID_j(\omega^i) + \gamma)}{\prod_j (w_j(\omega^i) + \beta \Sigma_j(\omega^i) + \gamma)}$$

という更新を全行で満たすことを示す。最初と最後が 1 に戻れば、全体の積が一致したことになる。

6.9 lookup を式の気持ちで理解する

lookup は、「この値は許可された表の中にある」という制約である。例えば byte 値であることを示したいなら、

$$v \in \{0, 1, \dots, 255\}$$

を証明したい。これを bit 分解で表すこともできるが、lookup table を使えば「 v が byte table に含まれる」として扱える。ハッシュ関数、範囲チェック、VM 命令の opcode など、表で確認したい処理に向いている。

lookup も、多重集合の一致として理解できる。例えば、証明中で出てくる値の列を

$$v_0, v_1, \dots, v_{r-1}$$

とし、許可された table を

$$T = \{0, 1, \dots, 255\}$$

とする。証明したいことは、各 v_i が T の中に含まれることである。実際の lookup argument では、入力列と table 列を圧縮したり、並べ替えたり、余分な table 要素を追加したりして、「入力を使った値を table 側で受け止められる」ことを示す。

複数列の lookup もよく使う。例えば命令表が

opcode	x	y	z
1	x	y	$x + y$
2	x	y	$x \cdot y$

のような意味を持つとき、 (opcode, x, y, z) というタプルが table に含まれることを示せば、VM の一命令を一つの lookup として扱える。この発想は、zkVM やハッシュ関数の実装で頻繁に使われる。

■注意

lookup は便利だが、table の意味を正しく設計する必要がある。例えば byte table を使っても、その値がどの整数表現の一部なのか、桁の順序は何か、overflow をどう扱うかは別の制約で決める。lookup は「表に含まれる」という主張を助ける道具であり、プログラム全体の意味づけは周辺の gate や copy constraint と合わせて決まる。

6.10 CCS

CCS は、Customizable Constraint Systems の略で、R1CS など一般化して扱うための枠組みである。特に folding scheme や Nova 系の文脈で現れる。CCS の目的は、さまざまな制約表現を、同じ形の「線形結合の積の和」として扱うことである。

R1CS では各制約が

$$\langle A, z \rangle \langle B, z \rangle = \langle C, z \rangle$$

という決まった形をしていた。CCS では、より一般に、複数の線形結合の積や和を組み合わせて制約を表せる。そのため、R1CS を特別な場合として含みつつ、folding scheme で扱いやすい形に整理できる。

6.11 CCS の定義

有限体を $\mathbb{F}$ とする。CCS では、まず次のデータを固定する。

- 行数 m と変数数 n 。
- 行列 $M_1, \dots, M_t \in \mathbb{F}^{m \times n}$ 。
- 項を指定する集合 $S_1, \dots, S_q$ 。各 S_i は $\{1, \dots, t\}$ の部分集合または多重集合である。
- 係数 $c_1, \dots, c_q \in \mathbb{F}$ 。

公開入力を x 、witness を w とし、定数 1 も含めて

$$z = (w, x, 1) \in \mathbb{F}^n$$

と置く。各行列 M_j は、変数ベクトル z から m 個の線形結合を作る。

$$M_j z \in \mathbb{F}^m$$

CCS の制約は、次のベクトル等式である。

$$\sum_{i=1}^q c_i \cdot \bigcirc_{j \in S_i} (M_j z) = 0 \in \mathbb{F}^m$$

ここで $\bigcirc$ は Hadamard 積、つまり成分ごとの積である。二つのベクトル $u, v \in \mathbb{F}^m$ について、

$$(u \circ v)_r = u_r v_r$$

である。したがって、

$$\bigcirc_{j \in S_i} (M_j z)$$

は、いくつかの線形結合を行ごとに掛け合わせたベクトルを意味する。成分ごとに書くと、各行 $r = 1, \dots, m$ について

$$\sum_{i=1}^q c_i \prod_{j \in S_i} (M_j z)_r = 0$$

が成り立つという意味である。各 S_i の大きさの最大値を degree と呼ぶことが多い。R1CS は degree 2 の典型例である。

6.12 R1CS は CCS の特別な場合

R1CS の制約は、行列 A, B, C を使って

$$(Az) \circ (Bz) - Cz = 0$$

と書ける。これは CCS で

$$M_1 = A, \quad M_2 = B, \quad M_3 = C$$

$$S_1 = \{1, 2\}, \quad S_2 = \{3\}$$

$$c_1 = 1, \quad c_2 = -1$$

と置いた場合である。実際、

$$c_1((M_1 z) \circ (M_2 z)) + c_2(M_3 z) = (Az) \circ (Bz) - Cz$$

となる。つまり CCS は、R1CS の「左の線形結合と右の線形結合を掛け、別の線形結合と等しい」という形を、より広い和と積の形へ拡張している。

6.13 Plonkish 風の gate も CCS で見る

Plonkish の標準的な gate

$$q_M a b + q_L a + q_R b + q_O c + q_C = 0$$

も、行ごとに見れば「線形結合の積の和」である。各行について、行列が次の値を取り出すように作られていると考える。

$$M_1 z = (q_M a)_i, \quad M_2 z = b_i, \quad M_3 z = (q_L a)_i, \quad M_4 z = (q_R b)_i, \quad M_5 z = (q_O c)_i, \quad M_6 z = (q_C)_i$$

すると、各行の制約は

$$(M_1 z) \circ (M_2 z) + (M_3 z) + (M_4 z) + (M_5 z) + (M_6 z) = 0$$

という CCS の形で表せる。ここで q_M, q_L, q_R, q_O, q_C は回路側で固定された selector なので、対応する行列の係数に組み込める。

より高い degree のカスタムゲートも同じ考え方で表せる。例えば

$$a_i b_i c_i - d_i = 0$$

という degree 3 の gate は、

$$(M_1 z) \circ (M_2 z) \circ (M_3 z) - (M_4 z) = 0$$

という形になる。CCS では、 S_i の大きさを変えることで、degree 2 の R1CS から高 degree gate まで同じ記法で扱える。

6.14 CCS が folding で役立つ理由

folding scheme では、二つの証明対象を一つの証明対象へまとめる。直感的には、「instance 1 が正しい」「instance 2 が正しい」という二つの主張を、ランダム係数で混ぜた一つの主張に圧縮する。これを繰り返すと、長い計算の各 step を少しずつ畳み込める。

このとき、制約が共通の形式で書かれていると扱いやすい。CCS では、制約が

$$\sum_i c_i \cdot \prod_{j \in S_i} \text{linear form}_{i,j}(z) = 0$$

という形にそろっている。各項は「線形結合を作る」「同じ行どうして掛ける」「係数を掛けて足す」という操作に分かれている。この構造により、R1CS、Plonkish 風の gate、高 degree gate を、folding 用の共通インターフェースへ持ち込みやすくなる。

■注意

CCS の役割は、計算をどのような制約として表すかを定める arithmetization 形式である。短い proof を作る段階では、commitment scheme、sumcheck、folding scheme などのプロトコル部品と組み合わせる。

6.15 比較表

方式	主な表現	得意な計算	よく出る文脈	初学者向けの理解
R1CS	乗算制約の集合	算術回路	Groth16、Circom	回路を制約に分解する
AIR	実行トレースと遷移	VM、繰り返し計算	STARK	計算の履歴が正しいか見る
Plonkish	gate と copy constraint	汎用回路、lookup	Plonk 系	表計算のように制約を書く
CCS	一般化された制約	folding-friendly な表現	Nova 系	複数方式をまとめる抽象化

6.16 同じ計算を複数の見方で見ると

$x^3 + x + 5 = y$ という同じ計算でも、arithmetization によって見方が変わる。

- R1CS では、掛け算を中間変数へ分解する。
- AIR では、計算を step ごとの状態遷移として表すこともできる。
- Plonkish では、各行の gate や selector として表に配置する。
- CCS では、制約の形をより一般化して扱う。

変わらないのは、「公開入力と秘密入力を分け、計算が正しく行われたことを有限体上の制約で示す」という目的である。

■確認問題

1. arithmetization を一文で説明せよ。
2. R1CS で $x^3 + x + 5 = y$ を表すとき、なぜ中間変数が必要になるか。
3. AIR で boundary constraint と transition constraint を分ける理由を説明せよ。
4. Plonkish arithmetization で lookup が役立つ例を一つ考えよ。
5. CCS の式で、R1CS がどのように表されるか説明せよ。

■まず覚えること

- arithmetization は、計算を有限体上の制約へ翻訳する工程である。
- Plonkish では、行、列、gate、selector、copy constraint、lookup を組み合わせて回路を表す。
- CCS では、複数の線形結合の Hadamard 積を足し合わせる形で制約を統一的に書く。
- 開発者が書く API と、証明システムが扱う制約の間にはコンパイル過程がある。

7 講義内チェックリスト

講義中の理解度チェックは、小テストとして採点するよりも、章末で短い問いを投げ、周囲と 30 秒話してから答えを共有する形にするとよい。

セクション	確認質問	期待する答えの方向性
zkSNARK 導入	witness と statement の違いは何か	witness は主張を成り立たせる補助情報、statement は公開された主張
有限体	なぜ有限体上で計算するのか	計算を有限集合内で代数的に扱い、制約や多項式に載せるため
多項式、FFT	FFT は何を高速化するのか	係数表現と評価表現の変換、または多点評価
楕円曲線	離散対数問題は何を難しくしているのか	P と xP から x を戻すこと
Commitment	binding と hiding の違いは何か	変更不能性と秘匿性
Arithmetization	R1CS と AIR の見方の違いは何か	回路制約として見るか、実行トレースとして見るか

8 ホワイトボードセッション

8.1 目的

ホワイトボードセッションの目的は、概念を他の参加者に向けて説明することで、理解を受け身から能動的なものに変えることである。調べる、整理する、図にする、質問に答えるという流れを通じて、自分が何を理解していないかを発見する。

8.2 推奨進行

時間	活動	成果物
0:00–0:15	個別調査	メモ
0:15–0:30	グループ議論と役割分担	調査分担
0:30–1:00	個別深掘り	説明素材
1:00–1:15	AI で 1 枚 HTML レポート作成	1 ページレポート
1:15–2:30	ピアレビュー	改善メモ、未解決の問い

8.3 グループ内の役割

- ファシリテーター: 時間管理と論点整理を行う。

- リサーチャー: 参考資料を読み、根拠を確認する。
- ダイアグラム担当: 図や例を作る。
- レポート担当: HTML レポートにまとめる。
- 発表担当: ピアレビュー時に説明する。

人数が少ない場合は、ファシリテーターと発表担当、リサーチャーとレポート担当を兼任してよい。

8.4 1 枚 HTML レポートの構成

1. タイトル
2. 30 秒で言うと
3. なぜ重要か
4. 主要概念
5. 図またはインタラクティブ要素
6. 具体例
7. よくある誤解
8. 参考資料
9. 未解決の問い

8.5 ピアレビュー観点

観点	期待水準	よくある不足
正確性	主要概念を大きく誤解していない	用語だけ合っていて、対象や仮定が混ざっている
説明力	初学者が 30 秒で要点を掴める	詳細は多いが、何が嬉しいかが不明
図解	図が概念間の関係を明確にしている	装飾的で、理解に貢献していない
接続性	zkSNARK 全体のどこに使うかが示されている	単独トピックとして閉じている
誠実さ	未解決の問いや不確かな点を分けている	推測と事実が同じ強さで書かれている

9 実装課題の選び方

講義とホワイトボードで得た理解は、コード、可視化、小さな攻撃、小さな証明のいずれかで確認すると定着しやすい。各参加者が興味と経験に合う課題を選ぶ。興味と経験に応じて、次のように選ぶ。

参加者の状態	推奨課題	理由
zk も実装もこれから	課題 C	witness と constraint の関係を最短で掴める
数学の直感をコードで確認したい	課題 D	trace と遷移制約が視覚化しやすい
暗号プロトコルに興味がある	課題 A	不十分な検証条件の危険性を体験できる
Ethereum 文脈に関心がある	課題 B	KZG の実用上の使われ方に接続できる
DSL や開発者体験に興味がある	課題 E	arithmetization を API 設計として考えられる

9.1 各 README に入れる項目

実装課題の README には、基本項目として次を入れる。

- 課題のゴール。
- 先に知っておくとよい概念。
- セットアップ手順。
- 最初に実行するコマンド。
- 正常系と異常系の期待結果。
- 参加者が編集するファイル。
- 詰まった時のヒントへのリンク。
- 発展課題。

9.2 課題 A: 不十分な KZG protocol に対する攻撃

- 狙い: KZG の仕組みと、検証条件不足の危険性を理解する。
- 成果物: 攻撃コード、攻撃が成功するテスト、修正方針、学習メモ。
- 完了条件: なぜ元の protocol が不十分かを、検証式または検証ロジックに紐づけて説明できる。

9.3 課題 B: Ethereum blob で使われる commitment scheme を調べる

- 狙い: KZG が実システムでどう使われるか、on-chain verification にどんな制約があるかを理解する。
- 成果物: Solidity contract、テスト、blob/KZG の関係図、gas や制約に関するメモ。
- 完了条件: 正常な proof が通り、不正な proof または不正な入力が失敗する。Ethereum では EVM が blob 本体を直接読まず、versioned hash と point evaluation precompile を通じて

KZG proof を検証する点も説明できる。

9.4 課題 C: 3 次方程式の解を知っていることを証明する回路を書く

- 狙い: witness、public input、constraint の関係を手で確認する。
- 成果物: 回路コード、witness 生成例、proof 生成または制約チェック、制約分解メモ。
- 完了条件: x を秘密、 y を公開入力として分け、不正な入力で検証が失敗する。

9.5 課題 D: 簡易ステートマシンを AIR で実装する

- 狙い: AIR の実行トレースと遷移制約の考え方を理解する。
- 成果物: trace 生成コード、遷移制約チェック、正常 trace と不正 trace のテスト、R1CS との比較。
- 完了条件: trace table の各列、遷移制約、boundary constraint を説明できる。

9.6 課題 E: Arithmetization API のプロトタイプ

- 狙い: 人間にとって書きやすい制約記述 API を考える。
- 成果物: API プロトタイプ、使用例、展開された制約の表示、API デザインの判断メモ。
- 完了条件: 公開入力と秘密入力の違い、生成される制約、隠した詳細と隠すと危険な詳細を説明できる。

10 用語集

statement 公開された主張。検証者が確認したい対象。

witness 主張を成り立たせる補助情報。zk では通常、検証者に隠したい入力として扱う。

proof / argument witness を直接明かさず、statement が正しいことを示す証拠。SNARK では計算量的な知識健全性に基づく argument と呼ぶのが厳密である。

SNARK Succinct Non-interactive Argument of Knowledge の略。短い非対話な argument で、知識健全性を持つ。

有限体 有限個の要素からなり、足し算、引き算、掛け算、0 以外での割り算ができる代数構造。

$\mathbb{Z}/m\mathbb{Z}$ 整数を m で割った余りの集合。足し算、引き算、掛け算が閉じた剰余環。

$\mathbb{F}_p$ 素数 p に対する剰余体。 $\mathbb{Z}/p\mathbb{Z}$ と同じ集合を体として見たもの。

$\mathbb{F}_p^*$ $\mathbb{F}_p$ から 0 を除いた集合。掛け算について巡回群になる。

$\langle g \rangle$ 元 g が生成する巡回群。

DL / CDH / DDH 離散対数問題、計算 Diffie–Hellman 問題、判定 Diffie–Hellman 問題。巡回群上の代表的な安全性仮定。

E/k 体 k 上の楕円曲線。典型的には $y^2 = x^3 + ax + b$ と無限遠点からなる。

$E[m]$ 楕円曲線の m 等分点、またはねじれ点の集合。 $mP = O$ を満たす点全体。

ペアリング $e : G_1 \times G_2 \rightarrow G_T$ のような双線形写像。KZG などでは隠れたスカラー関係を検証するために使う。

多項式 commitment 多項式を短い値に固定し、特定点での評価を後で証明できる commitment。

binding commit 後に別の値へ開けない性質。

hiding commitment から値が漏れない性質。

SRS Structured Reference String の略。KZG などを使う公開パラメータ。生成時の秘密が残ると安全性に影響する。

arithmetization 計算を有限体上の制約へ翻訳する工程。

R1CS Rank-1 Constraint System の略。線形結合どうしの積が別の線形結合に等しい、という制約の集合として計算を表す方式。

AIR 実行トレースと遷移制約として計算を表す方式。

Plonkish gate、wire、selector、copy constraint などでは計算を表す方式。

CCS Customizable Constraint Systems の略。複数の制約表現を一般化して扱う枠組み。

11 Further Reading

11.1 入門

- RareSkills ZK Book: <https://rareskills.io/zk-book>
- [MoonMath Manual PDF](#)
- [SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge](#)

11.2 有限体、多項式、FFT

- 有限体の四則演算を小さな素数で実装する記事またはノート。
- 多項式補間と FFT を可視化した教材。
- STARK や Plonk の資料で、FFT がどこに使われるかを示す節。

11.3 Commitment Scheme

- Merkle tree の実装チュートリアル。
- Pedersen commitment の加法準同型性を説明する資料。
- [Constant-Size Commitments to Polynomials and Their Applications](#)
- [EIP-4844: Shard Blob Transactions](#)

11.4 Arithmetization

- Circom の入門資料。
- Plonkish arithmetization の gate と selector の説明。
- AIR/STARK の実行トレース入門。

- [Nova: High-speed recursive arguments from folding schemes](#)

12 確認問題の解答例

12.1 zkSNARK 導入

1. witness は主張を成り立たせる補助入力、statement は公開された主張である。zk では witness を非公開に保つことが多い。
2. on-chain verification、軽量クライアント、プライバシーを保った認証などでは、検証を短く安く済ませたい。
3. 伝えたいのは statement が正しいこと。隠したいのは witness や、witness から推測できる余計な情報である。

12.2 有限体、多項式、FFT

1. $\mathbb{F}_7$ では、 $6 + 5 = 11 \equiv 4$ 、 $4 \cdot 5 = 20 \equiv 6$ 、 $3^{-1} = 5$ 。
2. 次数 d 以下の多項式を一意に決めるには、異なる $d + 1$ 点が必要である。
3. FFT は、係数表現と評価表現の変換、または多点評価と補間を高速化する。

12.3 楕円曲線

1. 離散対数問題。
2. 乱数 r によって同じ値でも異なる commitment にでき、値 m を隠すため。
3. commit された多項式の評価関係を、短い proof で検証するために使われる。

12.4 Commitment Scheme

1. binding は後から別の値に変えられない性質、hiding は開示前に値が漏れない性質である。
2. root までの経路にある sibling hash だけを示せばよく、木の高さが $\log n$ だからである。
3. 値を開かずに合計や残高更新の関係を扱う場面で役立つ。
4. opening proof が定数サイズで、多項式の特定点での値を短く証明できる。

12.5 Arithmetization

1. arithmetization は、計算を有限体上の制約へ翻訳する工程である。
2. R1CS の各制約は基本的に一つの乗算を扱うため、 x^2 や x^3 を中間変数に分ける必要がある。
3. transition constraint は各 step の関係を、boundary constraint は始点や終点など外側の条件を表すため。
4. byte range check、固定テーブルからの値選択、ハッシュ関数内部の小さな演算表などで役立つ。

5. CCS で $M_1 = A, M_2 = B, M_3 = C$ 、 $S_1 = \{1, 2\}, S_2 = \{3\}$ 、 $c_1 = 1, c_2 = -1$ と置くと、制約は $(Az) \circ (Bz) - Cz = 0$ となり、R1CS と同じ形になる。